

# SOCIAL NETWORKS

NPTEL Online Course • Prof. Sudarshan Iyengar, IIT Ropar

## Week 6 — Question Paper (Comprehension-Based MCQ / MSQ)

---

### Comprehension 1 — Ranking Influence: From Endorsements to Importance

#### Case Study

A media-analytics team wants to rank social-media accounts by genuine influence. They picture the accounts as a graph: each account is a node, and a public endorsement one account shouting out or recommending another is an edge. Because endorsements are one-directional acts, the graph is directed. To gather it, a program follows endorsements at random, hopping from account to account; this random-walk sampling reaches essentially the whole reachable graph after only a modest number of steps, growing roughly like  $n \cdot \log n$  for  $n$  accounts, so collecting the structure is entirely practical.

To turn the structure into a ranking, the team studies two scoring games. In the equal-sharing game, every account starts with the same pile of points and, each round, splits its current points equally among the accounts it endorses; after many rounds the piles settle, and the accounts are ranked by their final points. In the random-walk-drop game, a surfer roams the graph at random and drops one coin on each account it lands on; after very many steps, the accounts are ranked by coins collected. Remarkably, the two games rank the accounts in the same order.

The ranking captures a recursive notion of influence: an account is influential when other influential accounts endorse it not merely when many accounts do. An account therefore gains standing from being endorsed by high-scoring accounts that spread their endorsements across only a few others. The team also checks this against raw popularity and finds that an account's endorsement count and its influence score need not move together at all.

#### Questions 1–10

**Q1. [MCQ]** Why is the endorsement graph treated as a directed graph?

- A. An endorsement points one way: account X endorsing Y does not mean Y endorses X back.
- B. Every endorsement is mutual, so the resulting graph is undirected by construction.
- C. Endorsements count as directed only when the two accounts are on the same platform.
- D. The graph is directed because each account endorses exactly one other account in total.

**Answer: A**

**Reference:** Lecture 75 – The Web Graph (≈07:35–end)

**Solution:** An endorsement is one-sided: you can endorse an account that does not endorse you back, so the relationship is asymmetric and the edge has a direction. Endorsements are not mutual, are not restricted to same-platform accounts, and an account can endorse many others, so the other options misdescribe the graph.

---

**Q2. [Match the following]** Match each idea in Column A with its description in Column B.

| Column A (idea)          | Column B (description)                                                   |
|--------------------------|--------------------------------------------------------------------------|
| 1. Endorsement graph     | a. drop one coin on each node that a random walk visits                  |
| 2. Sampling (crawling)   | b. accounts are nodes and endorsements are directed edges                |
| 3. Equal-sharing game    | c. explore the graph by repeatedly following a random endorsement        |
| 4. Random-walk-drop game | d. each account splits its points equally among the accounts it endorses |

- A. 1-b, 2-c, 3-a, 4-d
- B. 1-c, 2-b, 3-d, 4-a
- C. 1-b, 2-c, 3-d, 4-a
- D. 1-a, 2-d, 3-b, 4-c

**Answer: C (1-b, 2-c, 3-d, 4-a)**

**Reference:** Lectures 75–78 – Web graph, crawling, and the two scoring games

**Solution:** The endorsement graph has accounts as nodes and directed endorsements as edges (1-b). Sampling explores it by following endorsements at random (2-c). The equal-sharing game has each account divide its points among the accounts it endorses (3-d), while the random-walk-drop game drops a coin on each visited account (4-a).

**Q3. [MCQ]** Which statement best captures the notion of influence behind the ranking?

- A. An account is influential if it endorses a large number of other accounts itself.
- B. An account is influential purely from how many accounts endorse it, whichever they are.
- C. An account is influential if it repeats a trending hashtag many times over.
- D. An account is influential if other influential accounts endorse it — a recursive notion.

**Answer: D**

**Reference:** Lecture 78 – Random Coin Dropping (≈05:19–end)

**Solution:** Influence is defined recursively: you are influential if influential accounts endorse you, which in turn depends on who endorses them. Sheer counts of endorsements received or sent, and hashtag repetition, all miss this recursive, importance-weighted character.

**Q4. [MCQ]** What is the relationship between the equal-sharing game and the random-walk-drop game?

- A. The two games usually disagree, so the team runs both and averages their orders.
- B. Equal-sharing and random-walk-drop rank the accounts in exactly the same order.
- C. Only the equal-sharing game yields a usable ranking; the walk game does not rank.
- D. The random-walk game ranks the accounts, while equal-sharing only counts the edges.

**Answer: B**

**Reference:** Lecture 78 – Random Coin Dropping (≈03:59–end)

**Solution:** The two games are different procedures but produce the same ordering of accounts, which is why either can be used to compute the ranking. They do not generally disagree, and both genuinely rank the accounts rather than merely counting edges.

---

**Q5. [MCQ]** Three accounts start with 100 points each. A endorses B and C; B endorses C; C endorses A. After one round of equal sharing, how many points does C hold?

- A. 50
- B. 100
- C. 150
- D. 200

**Answer: C**

**Reference:** Lecture 77 – Equal Coin Distribution (≈02:37–end)

**Solution:** A splits its 100 points over its two endorsements, sending 50 to each of B and C; B sends all 100 to C; C sends its 100 to A. So C receives 50 (from A) + 100 (from B) = 150, while A ends with 100 and B with 50.

---

**Q6. [MCQ]** Roughly how many random-walk steps does it take to visit every node of an  $n$ -node graph?

- A. Roughly a small multiple of  $n$  (about  $n \cdot \log n$ ) random steps cover all the nodes.
- B. About  $n^2$  random steps are required before every node has been visited once.
- C. It is generally impossible to reach every node just by walking at random.
- D. Exactly  $n$  random steps suffice to visit all  $n$  nodes, with none ever repeated.

**Answer: A**

**Reference:** Lecture 76 – Collecting the Web Graph (≈09:13–end)

**Solution:** Experiments showed the number of steps to cover the graph grows close to  $n \cdot \log n$  — a small multiple of  $n$ , looking nearly linear. It is not quadratic, it is very much possible, and it certainly takes more than exactly  $n$  steps because the walk revisits nodes along the way.

---

**Q7. [MCQ]** When the team compares each account's endorsement count (DegreeRank) with its influence score (PageRank), what do they find?

- A. Endorsement count and influence are tightly correlated, so either measure alone is enough.
- B. An account with a high endorsement count always has a correspondingly high influence score.
- C. An account with a low endorsement count can never reach a high influence score at all.
- D. They aren't correlated: many weak endorsements can lose to a few important ones.

**Answer: D**

**Reference:** Lecture 86 – Degree Rank versus Page Rank (≈05:55–end)

**Solution:** Just as on the citation network, the two measures need not line up: an account endorsed by many unimportant accounts can score low, while one endorsed by a few influential ones can score high. So endorsement count and influence are not correlated, and neither a high nor a low count fixes the score.

---

**Q8. [MCQ]** In the scoring games, what makes an account accumulate a large number of points?

- A. An account gains the most points by endorsing as many other accounts as it possibly can.
- B. An account gains points when high-point accounts that endorse few others endorse it.
- C. An account gains points mainly in proportion to how many endorsements it sends out.
- D. An account gains points only when low-point accounts happen to be the ones endorsing it.

**Answer: B**

**Reference:** Lecture 78 – Random Coin Dropping (≈07:28–end)

**Solution:** An account does well when rich backers endorse it and those backers spread their points over only a few endorsements, so each one carries a large share — like a wealthy relative with few people to help. Sending many endorsements does not raise your own score, and points flowing from low-scoring accounts contribute little.

**Q9. [MSQ]** Which statements about the endorsement graph and sampling are correct? (Select all that apply.)

- I. The endorsement graph is directed because endorsements point one way.
  - II. Sampling the graph is modelled as a random walk over the endorsement graph.
  - III. When the top accounts are listed, they are ordered by their accumulated points.
  - IV. An account's rank is determined only by how many posts the account has made.
- A. I, II, IV
  - B. II, III, IV
  - C. I, II, III
  - D. I, IV

**Answer: C (statements I, II, III)**

**Reference:** Lectures 75–79 – Web graph, crawling, and the ranking

**Solution:** The endorsement graph is directed (I), sampling is a random walk over it (II), and accounts are listed in order of the points each has accumulated (III). An account's rank comes from the endorsement structure, not from how many posts it has made, so IV is false.

**Q10. [MSQ]** Which statements about the ranking idea are correct? (Select all that apply.)

- I. The equal-sharing and random-walk games rank the accounts the same way.
  - II. Influence is recursive: influential accounts are those endorsed by influential accounts.
  - III. An account's endorsement count and its influence score need not be correlated.
  - IV. The account with the highest endorsement count always has the highest influence score.
- A. I, II, III
  - B. I, II, IV
  - C. II, III, IV
  - D. I, IV

**Answer: A (statements I, II, III)**

**Reference:** Lectures 77–78, 86 – the two games and DegreeRank vs PageRank

**Solution:** The two games agree on the order (I), influence is recursive (II), and endorsement count and influence score can diverge (III). Because they can diverge, the most-endorsed account is not guaranteed the highest score, so IV is false.

---

## Comprehension 2 — Implementing the Influence Ranking in NetworkX

### Case Study

A developer ranks social-media accounts by influence, implementing PageRank by the points-distribution method on a directed graph built in NetworkX. Each node is an account, and an edge  $i \rightarrow j$  means account  $i$  endorses account  $j$ . The graph is created with `nx.DiGraph`, and edges are added probabilistically: for each ordered pair of distinct nodes a random number in  $[0,1]$  is drawn, and the edge is added when that number is at most a chosen probability  $p$ . Every node is then initialised with 100 points.

Each iteration redistributes the points: a node looks at its out-links (the accounts it endorses) and sends an equal share of its current points along each one, so a node holding  $P$  points with  $d$  out-links sends  $P/d$  to every account it endorses. This repeats until the point totals stop changing. One snag is a dangling node an account that endorses no one: it keeps receiving points but never passes any on, so points pile up there and the ranking is thrown off. The fix is a damping step: each round, every node holds back most of its points while a fixed fraction is pooled across the network and shared back equally among all nodes.

A second implementation uses the random-walk method: a surfer hops to a random endorsed account and drops a point on each account it visits, and when it reaches a dangling node it teleports to a uniformly random account and carries on. Either way, the accounts are finally sorted by their points to give the ranking, which is checked against NetworkX's built-in pagerank. Key fragments:

```
import networkx as nx, random, numpy as np

G = nx.DiGraph()
# add an edge i -> j (i endorses j) with probability p
r = random.random()
if r <= p:
    G.add_edge(i, j)

# one distribution step: send an equal share along each out-link
out = G.out_edges(i)
share = float(previous_points[i]) / len(out)
for e in out:
    new_points[e[1]] += share          # e[1] is the endorsed account

order = np.argsort(-np.array(points)) # accounts, highest points first
```

### Questions 1–10

**Q1. [MCQ]** Why is the graph built as a directed graph (`nx.DiGraph`)?

- A. An undirected graph is used, since points flow equally both ways along each edge.
- B. A directed graph is used, since points flow only along each node's out-links.
- C. A directed graph is used purely to make the drawing of the network easier to read.
- D. The choice of graph type makes no real difference to how the points are distributed.

**Answer: B**

**Reference:** Lecture 80 – Points Distribution Method 1 (≈01:49–end)

**Solution:** PageRank sends a node's points along its out-links (its endorsements), so direction is essential — a node's outgoing edges decide where its points go. The graph type is not a cosmetic choice, and points do not flow both ways as they would on an undirected edge.

**Q2. [MCQ]** A node holding 100 points endorses 4 accounts (4 out-links). In one distribution step, how much does each endorsed account receive from it?

- A. 100, the node's full pile of points, handed to each of the endorsed accounts.
- B. 4, a single point for each of the node's four out-links in turn.
- C. 400, the node's points multiplied by its number of out-links.
- D. 25, the node's points divided equally among its 4 out-links.

**Answer: D**

**Reference:** Lecture 80 – Points Distribution Method 1 (≈00:06–01:00)

**Solution:** Each endorsed account gets an equal share,  $P$  divided by the number of out-links:  $100 / 4 = 25$ . The node does not hand its whole pile to each account, nor scale its points up by the link count.

**Q3. [MCQ]** In the distribution step the code iterates over `G.out_edges(i)`. Which description is correct?

- A. `G.out_edges(i)` lists  $i$ 's out-links; for each edge, index 1 is the account that gains the share.
- B. `G.out_edges(i)` lists  $i$ 's in-links; for each edge, index 0 is the account that gains the share.
- C. `G.out_edges(i)` returns just the count of  $i$ 's out-links as a single integer value.
- D. `G.out_edges(i)` works on undirected graphs and returns every edge touching node  $i$ .

**Answer: A**

**Reference:** Lecture 81 – Points Distribution Method 2 (≈12:00–end)

**Solution:** On a directed graph `G.out_edges(i)` returns the edges leaving  $i$ ; each edge is a (source, target) pair, so index 0 is  $i$  itself and index 1 is the endorsed account that receives the share. It is not a count, not the in-links, and is meant for directed graphs.

**Q4. [MCQ]** What is the “sink” problem that can distort the points-distribution ranking?

- A. A node with many out-links drains all of the points out of the whole network.
- B. A node with many in-links gives its points away faster than it can receive them.
- C. A node that endorses no one hoards the points it receives and never passes them on.
- D. Two nodes with equal points make the iteration oscillate and never converge.

**Answer: C**

**Reference:** Lecture 82 – Points Distribution Method 3 (≈11:15–end)

**Solution:** A dangling node — an account that endorses no one — keeps collecting points from its in-links but has nowhere to send them, so points accumulate there (or in a small group of such nodes) and the ranking is distorted. Out-links spread points rather than draining them, and equal points do not by themselves cause oscillation.

**Q5. [MCQ]** How does the damping step fix the sink problem?

- A. Every node passes on 100% of its points, so that nothing is ever held back at all.
- B. Each node keeps 80% of its points, and the pooled 20% is shared equally among all nodes.
- C. Each node keeps 20% of its points, while the other 80% is simply deleted from the graph.
- D. Every dangling node is removed from the graph before the point distribution begins.

**Answer: B**

**Reference:** Lecture 83 – Points Distribution Method 4 (≈00:51–end)

**Solution:** With  $s = 0.8$ , each node retains 80% of its points and the collected 20% is redistributed equally to every node — like a tax that is shared back to all — so points cannot pile up forever at a dangling node. Points are conserved (not deleted), and the dangling nodes are kept in the graph, not removed.

**Q6. [MCQ]** In the random-walk method, what happens when the surfer reaches an account that endorses no one?

- A. On reaching such a dead-end node, the walk stops and the whole run ends right there.
- B. On reaching such a dead-end node, the walk steps back to the node it just came from.
- C. On reaching such a dead-end node, that node is deleted and the walk then carries on.
- D. On reaching such a dead-end node, the walk teleports to a uniformly random node.

**Answer: D**

**Reference:** Lecture 84 – Random Walk Method 1 (≈00:05–end)

**Solution:** A dangling node has nowhere to walk to, so the surfer teleports — jumps to a uniformly random account — and resumes from there. The run does not end, the surfer does not back-track, and the node is not deleted.

**Q7. [Arrange in order]** Arrange the steps of the points-distribution implementation in the correct order.

*Steps (in scrambled order):*

- (i) Initialise every node with 100 points.
  - (ii) Create a directed graph and add its edges probabilistically.
  - (iii) Sort the nodes by their final points to obtain the ranking.
  - (iv) Repeatedly distribute points along out-links (with sink handling) until they converge.
  - (v) Compare the resulting ranking against NetworkX's built-in pagerank.
- A.  $i \rightarrow ii \rightarrow iv \rightarrow iii \rightarrow v$
  - B.  $ii \rightarrow i \rightarrow iv \rightarrow iii \rightarrow v$
  - C.  $ii \rightarrow i \rightarrow iii \rightarrow iv \rightarrow v$
  - D.  $ii \rightarrow iv \rightarrow i \rightarrow iii \rightarrow v$

**Answer: B ( $ii \rightarrow i \rightarrow iv \rightarrow iii \rightarrow v$ )**

**Reference:** Lectures 80–83 – Points-distribution pipeline

**Solution:** First create the directed graph and its edges (ii), then give every node 100 points (i), then distribute points repeatedly with sink handling until convergence (iv), then sort nodes by final points (iii), and finally validate against the built-in pagerank (v).

**Q8. [MCQ]** The code ranks nodes with `np.argsort`. How does it obtain a highest-first (descending) order?

- A. `argsort` already sorts largest-first, so it gives the descending ranking directly.
- B. It sorts the points in ascending order and reads them straight from top to bottom.
- C. It runs `argsort` on the negated points, so the highest-point node comes out first.
- D. It runs `argsort` on the node labels and ignores the points entirely while sorting.

**Answer: C**

**Reference:** Lecture 82 – Points Distribution Method 3 (≈01:27–05:07)

**Solution:** `argsort` returns indices that sort the values in ascending order, so negating the points first makes the largest value sort to the front — giving the highest-point account rank one. Plain `argsort` is ascending, and sorting the labels would ignore the scores.

**Q9. [MCQ]** The implementation is validated against NetworkX's `pagerank`. What does `nx.pagerank(G)` return, and how is it used here?

- A. It returns a `{node: score}` dictionary; its items are sorted by value, descending, to rank.
- B. It returns the ranking already as a finished, ordered list of the nodes from best to worst.
- C. It returns a single number, namely the rank of the single top node in the graph.
- D. It returns the in-degree of every node, which is then used directly as the score.

**Answer: A**

**Reference:** Lecture 82 – Points Distribution Method 3 (≈07:00–end)

**Solution:** `nx.pagerank` returns a dictionary mapping each node to its PageRank value, not a ready-made ranking, so the code turns the items into (node, value) tuples and sorts them by value in descending order. It does not return a finished list, a single number, or the in-degree.

**Q10. [MSQ]** Which statements about the implementation are correct? (Select all that apply.)

- I. The total number of points is conserved from one iteration to the next.
  - II. With sink handling, the method's ranking matches NetworkX's `pagerank`.
  - III. The random-walk method uses teleportation to escape dangling nodes.
  - IV. A node that endorses no one distributes its points equally to all other nodes.
- A. I, II, IV
  - B. II, III, IV
  - C. I, III, IV
  - D. I, II, III

**Answer: D (statements I, II, III)**

**Reference:** Lectures 81–84 – Points-distribution and random-walk methods

**Solution:** Points are conserved across iterations (I), the sink-handled method matches the built-in `pagerank` (II), and the random-walk method teleports out of dangling nodes (III). A dangling node endorses no one, so in the distribution method it simply keeps its own points rather than sharing them with everyone — that sharing is the job of the damping step — so IV is false.

*End of Week 6 Question Paper — 20 questions • 2 comprehensions*